Task 2 – Example Answer with Python Code

```
#Task 2

from datetime import date
import math

def price_contract(in_dates, in_prices, out_dates,
out_prices, rate, storage_cost_rate, total_vol,
injection_withdrawal_cost_rate):
    volume = 0
    buy_cost = 0
    cash_in = 0
    last_date = min(min(in_dates), min(out_dates))

    # Ensure dates are in sequence
    all_dates = sorted(set(in_dates + out_dates))

    for i in range(len(all_dates)):
        # processing code for each date
        start_date = all_dates[i]

        if start_date in in_dates:
            # Inject on these dates and sum up cash flows
            if volume <= total_vol - rate:
                volume += rate

                # Cost to purchase gas
                buy_cost += rate *
in_prices[in_dates.index(start_date)]
                # Injection cost
                injection_cost = rate *
injection_withdrawal_cost_rate
                buy_cost += injection_cost
                print('Injected gas on %s at a price of
%s'%(start_date, in_prices[in_dates.index(start_date)]))

            else:
                # We do not want to inject when rate is
greater than total volume minus volume
                print('Injection is not possible on date %s
as there is insufficient space in the storage
facility'%start_date)
        elif start_date in out_dates:
```

```
            #Withdraw on these dates and sum cash flows
            if volume >= rate:
                volume -= rate
                cash_in += rate *
out_prices[out_dates.index(start_date)]
                # Withdrawal cost
                withdrawal_cost = rate *
injection_withdrawal_cost_rate
                cash_in -= withdrawal_cost
                print('Extracted gas on %s at a price of
%s'%(start_date, out_prices[out_dates.index(start_date)]))
            else:
                # we cannot withdraw more gas than is
actually stored
                print('Extraction is not possible on date
%s as there is insufficient volume of gas
stored'%start_date)

    store_cost = math.ceil((max(out_dates) -
min(in_dates)).days // 30) * storage_cost_rate
    return cash_in - store_cost - buy_cost

# Example usage of price_contract()
in_dates = [date(2022, 1, 1), date(2022, 2, 1), date(2022,
2, 21), date(2022, 4, 1)] #injection dates
in_prices = [20, 21, 20.5, 22]#prices on the injection days
out_dates = [date(2022, 1, 27), date(2022, 2, 15),
date(2022, 3, 20), date(2022, 6, 1)] # extraction dates
out_prices = [23, 19, 21, 25] # prices on the extraction
days
rate = 100000  # rate of gas in cubic feet per day
storage_cost_rate = 10000  # total volume in cubic feet
injection_withdrawal_cost_rate = 0.0005  # $/cf
max_storage_volume = 500000 # maximum storage capacity of
the storage facility
result = price_contract(in_dates, in_prices, out_dates,
out_prices, rate, storage_cost_rate, max_storage_volume,
injection_withdrawal_cost_rate)
print()
print(f"The value of the contract is: ${result}")
```

```
## Explaining the Methodology Adopted for this Task ##

# The given Python code implements a function
`price_contract` that calculates the profit or loss
obtained by
# undertaking trades on given dates for a contract
involving the buying, storing, and selling of natural gas
the
# storage cost of the gas, the injection/withdrawal. The
value of the contract is the profit or loss obtained by
# undertaking the trades on given dates. Play around with
the parameters and you'll be able to see this.
# In the end the intent for this function returns the value
of the contract.
#The function takes in eight inputs:
#- `in_dates`: A list of dates on which the gas is being
injected into the storage facility.
#- `in_prices`: A list of prices of gas on each of the
injection dates.
#- `out_dates`: A list of dates on which the gas is being
withdrawn from the storage facility.
#- `out_prices`: A list of prices of gas on each of the
withdrawal dates.
#- `rate`: The rate of gas in cubic feet per day.
#- `storage_cost_rate`: A fixed monthly fee to store the
gas
#- `total_vol`: The total volume of gas in cubic feet that
can be stored.
#- `injection_withdrawal_cost_rate`: The
injection/withdrawal cost of gas in dollars per cubic foot.

# The function first ensures that all the dates are in
sequence and sorted in ascending order. Then, it iterates
#over all the dates and calculates the cash flows on each
date. If the current date is an injection date, it
#injects gas into the storage facility and calculates the
cost to store the gas, the cost to purchase the gas,
#and the injection cost. If the current date is a
withdrawal date, it withdraws gas from the storage facility
and
#calculates the cash inflow from selling the gas, the cost
to store the remaining gas, and the withdrawal cost.
```

```
# Finally, the function returns the net profit or loss by subtracting the storage cost and the cost to purchase
#the gas from the cash inflow from selling the gas.

# The example usage of the `price_contract` function calculates the profit or loss for a contract that involves
#injecting gas on four different dates and withdrawing gas on four different dates, each with a different price.
#The other inputs such as the rate of gas, the storage cost rate, the total volume, and the injection/withdrawal
#cost rate are also provided. The output is printed to the console using an f-string.
```